DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

Academic Year: 2026-27	**Name of Student: Neev Rambhia**
Semester: V	**Student ID: 25206010**
Class / Div / Branch: T.E./ C / CSE(AI&ML)	**Roll No: 154**
Subject: WMD Lab	**Date of Performance: 30/09/2026**
Name of Instructor: Prof. Vijesh Nair	**Date of Submission: 14/10/2026**

Experiment No. 9

Aim: Deploy Spring Boot app to a cloud platform (Render or Heroku) or local server.

Lab Outcomes: Apply microservices, REST APIs, and deployment techniques for scalable applications.

Software Used: Java JDK17 or above, Spring Boot, Maven, Git & Github Account, Render Account.

Program:

CommonComponents.jsx

```
import React from 'react';


export function Button(props) {
 const {
  children,
  variant = 'primary',
  size = 'md',
  onClick,
  type = 'button',
  disabled = false,
  className = '',
  icon: Icon = null
```

```
} = props;

const baseClass = 'btn';
const variantClass = `btn-${variant}`;
const sizeClass = size !== 'md' ? `btn-${size}` : '';

return (
  <button
    type={type}
    onClick={onClick}
    disabled={disabled}
    className={`${baseClass} ${variantClass} ${sizeClass} ${className}`.trim()}
  >
    {Icon && <Icon className="btn-icon" size={16} />}
    <span>{children}</span>
  </button>
);
}

export function Badge(props) {
  const { children, type = 'default', className = '' } = props;
  return (
```

```
  <span className={`badge badge-${type} ${className}`.trim()}>
    {children}
  </span>
 );
}

export function PageHeader(props) {
 const { title, subtitle, action } = props;
 return (
  <div className="page-header">
   <div className="page-header-text">
    <h1 className="page-title">{title}</h1>
    {subtitle && <p className="page-subtitle">{subtitle}</p>}
   </div>
   {action && <div className="page-header-action">{action}</div>}
  </div>
 );
}

export function SectionHeader(props) {
 const { title, subtitle, action } = props;
 return (
```

```
  <div className="section-header">
    <div>
      <h2 className="section-title">{title}</h2>
      {subtitle && <p className="section-subtitle">{subtitle}</p>}
    </div>
    {action && <div className="section-action">{action}</div>}
  </div>
 );
}

export function LoadingState(props) {
 const { message = 'Loading campus data...' } = props;
 return (
  <div className="state-box">
    <div className="loading-spinner"></div>
    <h3 className="state-title">Fetching Data</h3>
    <p className="state-message">{message}</p>
  </div>
 );
}

export function EmptyState(props) {
```

```
  const {
    icon: Icon = null,
    title = 'No Records Found',
    description = 'There are no items matching your criteria.',
    action = null
  } = props;
  return (
    <div className="empty-state">
      <div className="empty-icon-wrapper">
        {Icon ? <Icon size={32} /> : ' 📋 '}
      </div>
      <h3 className="empty-title">{title}</h3>
      <p className="empty-description">{description}</p>
      {action && <div style={{ marginTop: '1.25rem' }}>{action}</div>}
    </div>
  );
}

export function ErrorState(props) {
  const { title = 'API Connection Error', message, onRetry, onSwitchMock } = props;
  return (
    <div className="state-box error-box">
```

```
    <h3 className="state-title">{title}</h3>

    <p className="state-message">{message}</p>

    <p className="error-tip">

     Please check your internet connection or verify that the Fixora API backend service is running.

    </p>

    <div className="error-actions">

     {onRetry && (

      <button onClick={onRetry} className="btn btn-primary btn-sm">

       Retry REST API Connection

      </button>

     )}

     {onSwitchMock && (

      <button onClick={onSwitchMock} className="btn btn-secondary btn-sm">

       Use Static mockData.js

      </button>

     )}

    </div>

   </div>

  );

}
```

Dockerfile

```
# ========================================

# Fixora Backend Dockerfile

# Multi-stage build for Spring Boot on Render

# ========================================


# Stage 1: Build with Maven

FROM eclipse-temurin:21-jdk AS build

WORKDIR /app


# Copy Maven wrapper and pom.xml first (layer caching)

COPY .mvn/ .mvn/

COPY mvnw pom.xml ./


# Download dependencies (cached layer)

RUN chmod +x mvnw && ./mvnw dependency:resolve -B


# Copy source code and build

COPY src/ src/

RUN ./mvnw clean package -DskipTests -B


# Stage 2: Runtime image
```

```
FROM eclipse-temurin:21-jre

WORKDIR /app


# Copy the built JAR

COPY --from=build /app/target/campusfix-backend.jar app.jar


# Render sets the PORT env variable

EXPOSE ${PORT:-8080}


# Run with Render's PORT

ENTRYPOINT ["java", "-jar", "app.jar"]
```

IssueController.java

```
package com.campusfix.controller;


import com.campusfix.model.Issue;

import com.campusfix.service.IssueService;

import org.springframework.http.HttpStatus;

import org.springframework.http.ResponseEntity;

import org.springframework.web.bind.annotation.*;


import java.util.List;
```

```
import java.util.Map;


@RestController

public class IssueController {


    private final IssueService issueService;


    public IssueController(IssueService issueService) {

        this.issueService = issueService;

    }


    // ==================== ROOT & HEALTH CHECK ====================


    @GetMapping("/")

    public ResponseEntity<Map<String, Object>> root() {

        return ResponseEntity.ok(Map.of(

                "service", "Fixora Spring Boot REST API",

                "status", "UP",

                "endpoints", List.of("/api/health", "/api/issues")

        ));

    }
```

```
@GetMapping({"/api/health", "/health"})
public ResponseEntity<Map<String, String>> healthCheck() {
    return ResponseEntity.ok(Map.of(
            "status", "UP",
            "service", "Fixora Spring Boot Backend",
            "message", "Fixora API is running successfully"
    ));
}

// ==================== ISSUE CRUD ENDPOINTS ====================

// GET /api/issues or /issues — Fetch all issues
@GetMapping({"/api/issues", "/issues"})
public ResponseEntity<List<Issue>> getAllIssues() {
    List<Issue> issues = issueService.getAllIssues();
    return ResponseEntity.ok(issues);
}

// GET /api/issues/{id} or /issues/{id} — Fetch single issue
@GetMapping({"/api/issues/{id}", "/issues/{id}"})
public ResponseEntity<Object> getIssueById(@PathVariable Long id) {
    return issueService.getIssueById(id)
```

```
        .<ResponseEntity<Object>>map(ResponseEntity::ok)
        .orElse(ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(Map.of("error", "Issue not found")));
}

// POST /api/issues or /issues — Create new issue
@PostMapping({"/api/issues", "/issues"})
public ResponseEntity<Issue> createIssue(@RequestBody Issue issue) {
   Issue created = issueService.createIssue(issue);
   return ResponseEntity.status(HttpStatus.CREATED).body(created);
}

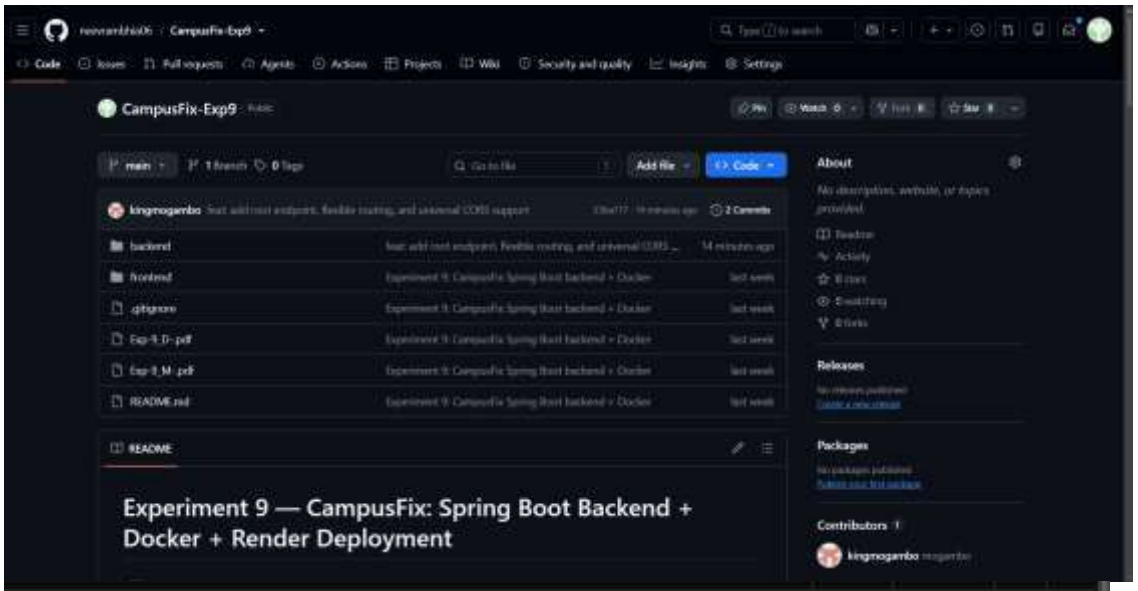
// PUT /api/issues/{id} or /issues/{id} — Replace entire issue
@PutMapping({"/api/issues/{id}", "/issues/{id}"})
public ResponseEntity<Object> updateIssue(@PathVariable Long id, @RequestBody Issue issue) {
   return issueService.updateIssue(id, issue)
        .<ResponseEntity<Object>>map(ResponseEntity::ok)
        .orElse(ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(Map.of("error", "Issue not found")));
}

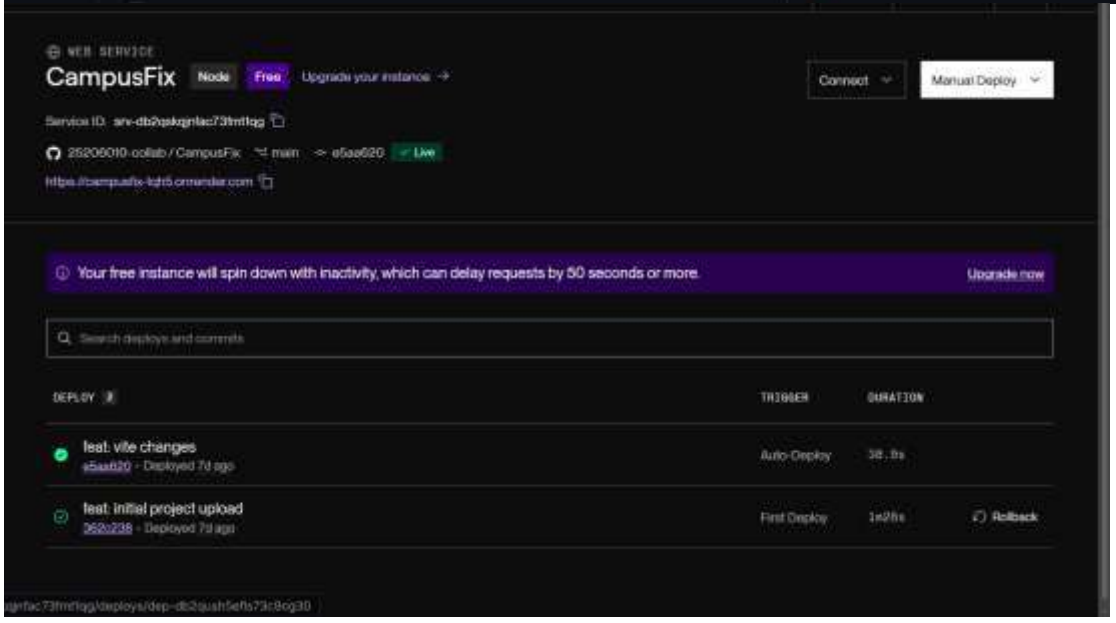
// PATCH /api/issues/{id} or /issues/{id} — Partial update
```

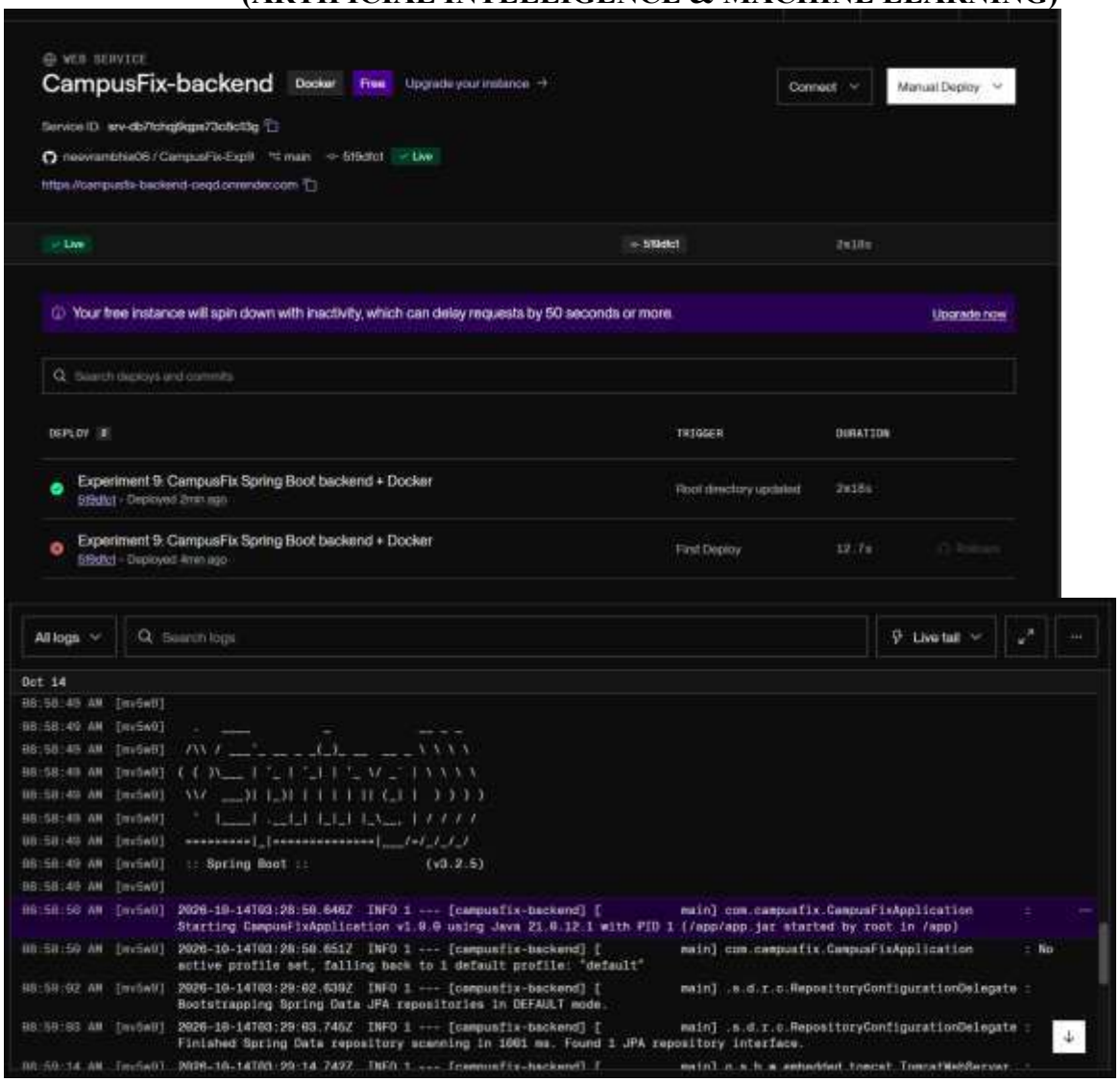
```
  @PatchMapping({"/api/issues/{id}", "/issues/{id}"})
  public ResponseEntity<Object> patchIssue(@PathVariable Long id, @RequestBody Issue patchData) {
    return issueService.patchIssue(id, patchData)
        .<ResponseEntity<Object>>map(ResponseEntity::ok)
        .orElse(ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(Map.of("error", "Issue not found")));
  }

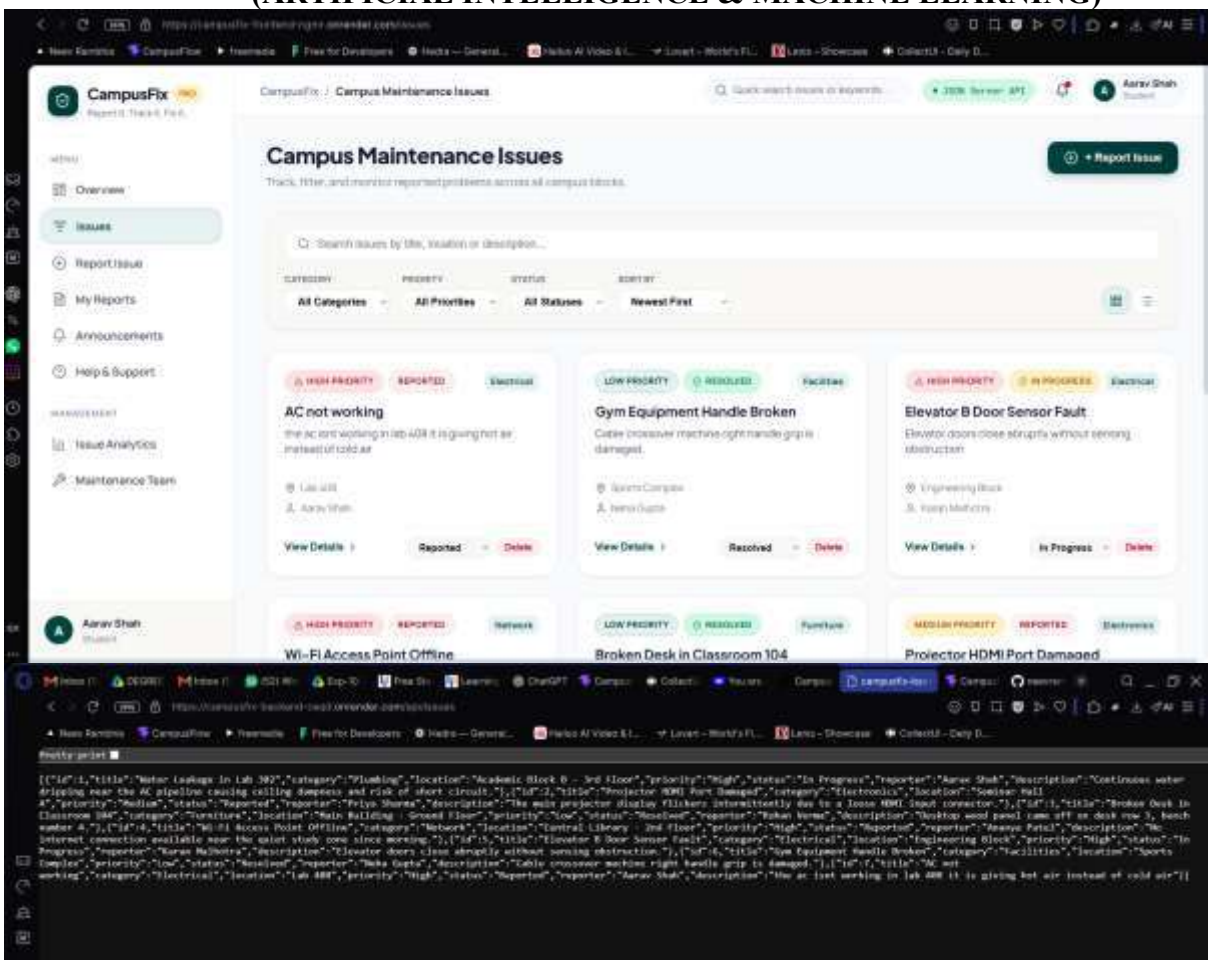
  // DELETE /api/issues/{id} or /issues/{id} — Delete issue
  @DeleteMapping({"/api/issues/{id}", "/issues/{id}"})
  public ResponseEntity<Object> deleteIssue(@PathVariable Long id) {
    if (issueService.deleteIssue(id)) {
      return ResponseEntity.ok(Map.of("success", true, "id", id));
    }
    return ResponseEntity.status(HttpStatus.NOT_FOUND)
        .body(Map.of("error", "Issue not found"));
  }
}
```

Output:


CampusFix-Exp9
main
Add file
Code
About
README
Experiment 9 — CampusFix: Spring Boot Backend + Docker + Render Deployment
Releases
Packages
Contributors


WEB SERVICE
CampusFix
Node
Free
Upgrade your instance
Connect
Manual Deploy
Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more.
Upgrade now
DEPLOY
TRIGGER
DURATION
feat: vite changes
Auto-Deploy
feat: initial project upload
First Deploy
Rollback


WEB SERVICE
CampusFix-backend
Docker
Free
Upgrade your instance
Connect
Manual Deploy
Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more
Upgrade now
DEPLOY
TRIGGER
DURATION
Experiment 9: CampusFix Spring Boot backend + Docker
Root directory updated
Experiment 9: CampusFix Spring Boot backend + Docker
First Deploy
All logs
Live tail
Oct 14
:: Spring Boot ::
(v3.2.5)
Starting CampusFixApplication v1.0.0 using Java 21.0.12.1 with PID 1 (/app/app.jar started by root in /app)
active profile set, falling back to 1 default profile: "default"
Bootstrapping Spring Data JPA repositories in DEFAULT mode.
Finished Spring Data repository scanning in 1001 ms. Found 1 JPA repository interface.



Conclusion: Thus, we demonstrated and deployed Spring Boot application to a cloud platform called Render.